

Introduction

Dans cette expérimentation, nous avons utilisé **FlowiseAI** couplé à un modèle **GPT-4o-mini** afin de générer dynamiquement des requêtes **Cypher** à partir de questions en langage naturel sur un **graphe de connaissance** modélisé autour de projets, réunions, actions et personnes.

La méthodologie suivie a été :

- Création manuelle des données initiales dans Neo4j.
- Pose de questions en langage naturel via FlowiseAI.
- Analyse des requêtes Cypher générées.
- Exécution dans Neo4j et vérification du retour.

Chaque cas est documenté ci-dessous.

1. Création du graphe de connaissance

- **Nœuds :**
 - `Person {name}`
 - `Project {name}`
 - `Meeting {date}`
 - `Action {description}`
- **Relations :**
 - `(:Person)-[:ATTENDED]->(:Meeting)`
 - `(:Meeting)-[:HAS_ACTION]->(:Action)`
 - `(:Action)-[:CONCERNS]->(:Project)`

```

// Création des personnes
CREATE (:Person {name: 'Alice Dupont'}),
      (:Person {name: 'Bob Martin'}),
      (:Person {name: 'Charlie Durand'});

// Création des projets
CREATE (:Project {name: 'Projet Apollo'}),
      (:Project {name: 'Projet Borealis'});

// Création des réunions
CREATE (:Meeting {date: date('2025-01-15')}),
      (:Meeting {date: date('2025-02-20')}),
      (:Meeting {date: date('2025-03-25')});

// Création des actions
CREATE (:Action {description: "Lancer l'étude de faisabilité du projet Apollo"}),
      (:Action {description: "Finaliser la maquette du projet Borealis"}),
      (:Action {description: "Organiser la réunion de revue de projet Apollo"});

// Vérification de la création
MATCH (n:Person) RETURN n LIMIT 25;
MATCH p=()-[:>]() RETURN p LIMIT 25;

// Liens entre personnes et réunions
MATCH (alice:Person {name: 'Alice Dupont'}), (meeting1:Meeting {date: date('2025-01-15')})
CREATE (alice)-[:ATTENDED]->(meeting1);

MATCH (bob:Person {name: 'Bob Martin'}), (meeting1:Meeting {date: date('2025-01-15')})
CREATE (bob)-[:ATTENDED]->(meeting1);

MATCH (bob:Person {name: 'Bob Martin'}), (meeting2:Meeting {date: date('2025-02-20')})
CREATE (bob)-[:ATTENDED]->(meeting2);

MATCH (charlie:Person {name: 'Charlie Durand'}), (meeting2:Meeting {date: date('2025-02-20')})
CREATE (charlie)-[:ATTENDED]->(meeting2);

MATCH (alice:Person {name: 'Alice Dupont'}), (meeting3:Meeting {date: date('2025-03-25')})
CREATE (alice)-[:ATTENDED]->(meeting3);

MATCH (charlie:Person {name: 'Charlie Durand'}), (meeting3:Meeting {date: date('2025-03-25')})
CREATE (charlie)-[:ATTENDED]->(meeting3);

// Liens entre réunions et actions
MATCH (meeting1:Meeting {date: date('2025-01-15')}), (action1:Action {description: "Lancer l'étude de faisabilité du projet Apollo"})
CREATE (meeting1)-[:HAS_ACTION]->(action1);

MATCH (meeting2:Meeting {date: date('2025-02-20')}), (action2:Action {description: "Finaliser la maquette du projet Borealis"})
CREATE (meeting2)-[:HAS_ACTION]->(action2);

MATCH (meeting3:Meeting {date: date('2025-03-25')}), (action3:Action {description: "Organiser la réunion de revue de projet Apollo"})
CREATE (meeting3)-[:HAS_ACTION]->(action3);

// Liens entre actions et projets
MATCH (action1:Action {description: "Lancer l'étude de faisabilité du projet Apollo"}), (apollo:Project {name: 'Projet Apollo'})
CREATE (action1)-[:CONCERNS]->(apollo);

MATCH (action2:Action {description: "Finaliser la maquette du projet Borealis"}), (borealis:Project {name: 'Projet Borealis'})
CREATE (action2)-[:CONCERNS]->(borealis);

MATCH (action3:Action {description: "Organiser la réunion de revue de projet Apollo"}), (apollo:Project {name: 'Projet Apollo'})
CREATE (action3)-[:CONCERNS]->(apollo);

```

2. Questions posées et analyse

Question 1 :

Question : "Quelles actions ont été réalisées pour chaque projet, et à quelle date ?"

Requête Cypher générée :

```
MATCH (m:Meeting)-[:HAS_ACTION]->(a:Action)-[:CONCERNS]->(p:Project)
RETURN p.name AS project, a.description AS action, m.date AS meeting_date
```

Analyse :

- **Correcte :** Oui.
- **Pourquoi :** Suit exactement la structure du graphe de connaissance. Retourne pour chaque projet la liste des actions et la date de la réunion liée.

Retour obtenu :

- Projet Apollo : 2 actions à 2 dates différentes.
- Projet Borealis : 1 action.

Question 2 :

Question : "Montre-moi l'ensemble des réunions et des actions concernant un projet Apollo"

Requête Cypher générée :

```
MATCH (m:Meeting)-[:HAS_ACTION]->(a:Action)-[:CONCERNS]->(p:Project {name: 'apollo'})
RETURN m, a
```

Analyse :

- **Correcte :** Non.
- **Pourquoi :**
 - Mauvaise casse : dans les données, le projet s'appelle "**Projet Apollo**" et non "**apollo**".
 - De plus, le champ `name` doit être comparé avec `CONTAINS 'Apollo'` pour plus de souplesse.

Correction proposée :

```
MATCH (m:Meeting)-[:HAS_ACTION]->(a:Action)-[:CONCERNS]->(p:Project)
WHERE toLower(p.name) CONTAINS 'apollo'
RETURN m.date AS meeting_date, a.description AS action
ORDER BY m.date
```

Retour : Aucun résultat obtenu avec la requête incorrecte.

Question 3 :

Question : "Combien d'actions ont été réalisées pour chaque projet ?"

Requête Cypher générée :

```
MATCH (a:Action)-[:CONCERNS]->(p:Project)
RETURN p.name AS Project, COUNT(a) AS ActionsCount
GROUP BY p.name
```

Analyse :

- **Correcte :** Non.
- **Pourquoi :**
 - En Cypher, il n'existe **pas** de `GROUP BY`. Le regroupement est implicite à partir du `RETURN` utilisant une fonction d'agrégat comme `COUNT`.

Correction proposée :

```
MATCH (a:Action)-[:CONCERNS]->(p:Project)
RETURN p.name AS Project, COUNT(a) AS ActionsCount
ORDER BY ActionsCount DESC
```

Retour :

- Projet Apollo : 2 actions.
- Projet Borealis : 1 action.

Conclusion

L'expérience a montré que **FlowiseAI + GPT-4o-mini** est capable de générer des requêtes Cypher correctes à partir de questions naturelles **simplement structurées**.

Cependant :

- Des écueils apparaissent lorsque le modèle doit gérer des détails précis (noms exacts, casse, syntaxe Cypher).
- Il est nécessaire de prévoir des **vérifications automatiques** et, si besoin, des **corrections** pour garantir une intégration fiable dans un environnement de production.

Améliorations envisageables :

- Utiliser une **normalisation automatique** (par exemple `toLowerCase`) pour les comparaisons de chaînes.
- Ajouter une validation systématique de la syntaxe Cypher générée.
- Coupler à un moteur de suggestions ou d'échantillons de questions pour éviter les pièges classiques.